

PHP in kleinen Schritten

16 Mini-Aufgaben • ein Formular • jeweils eine kleine Änderung

Das Formular und das Styling sind fertig. Du ergänzt pro Aufgabe nur den markierten PHP-Bereich. Arbeite eine Aufgabe nach der anderen ab und prüfe immer die angegebenen Testfälle.

Heute nicht: Sessions, Cookies, Datenbank, Klassen, verschachtelte Arrays oder eine große Mehrseiten-Anwendung. Hier geht es nur um Eingaben, einfache Arrays und kleine Kontrollstrukturen.

So startest du

1. Entpacke das Starterpaket vollständig. Starte PHP-START.cmd im Ordner php_kleine_schritte.
2. Öffne <http://127.0.0.1:8085/> im Browser. Das Serverfenster bleibt offen.
3. Wähle Aufgabe 01. Öffne daneben deren auswertung.php in VS Code.
4. Ergänze nur zwischen „AB HIER ARBEITEN“ und „BIS HIER ARBEITEN“. Speichere und sende das Formular erneut.

Pro Aufgabe dieselben drei Dateien

Datei	Was du damit machst
formular.html	Im Browser ausfüllen und absenden. Bis Aufgabe 15 nicht umbauen.
style.css	Ist fertig. Muss nicht bearbeitet werden.
auswertung.php	Hier ergänzt du den kleinen PHP-Arbeitsauftrag.

Wichtig: Die Aufgabenordner sind unabhängig. Benötigte Vorarbeiten stehen bereits in der jeweiligen PHP-Datei. Du musst keine gelöste Voraufgabe kopieren.

XAMPP-Alternative: Ordner nach C:\xampp\htdocs kopieren, Apache starten und http://localhost/php_kleine_schritte/ öffnen. Keine Datenbank nötig. Nicht über file:// oder per Doppelklick absenden.

Das Formular bleibt gleich

Vier Felder reichen. Die Standardwerte sind Mia, Paris und 2 Personen. Der Newsletter ist zunächst nicht angehakt.

Sichtbares Feld	name / Schlüssel	Übertragener Wert
Vorname	vorname	Mia oder dein Text; auch leer testbar
Reiseziel · Dropdown	ziel	berlin, paris, rom oder leer
Personen · Dropdown	personen	1, 2, 3, 4 oder leer; zunächst Text
Newsletter · Checkbox	newsletter	ja; ohne Haken fehlt dieser Schlüssel

Die Felder haben absichtlich kein `required`. Ein leerer Wert soll PHP erreichen können. **Leer ist nicht dasselbe wie fehlend.** Ein direkter Aufruf von `auswertung.php` sendet keine POST-Formulardaten.

GET und POST sind sichtbar vorbereitet

```
<form action="auswertung.php" method="post">
<form action="auswertung.php" method="get">
```

Diese zwei Zeilen sind Alternativen, nicht zwei Formulare in einer Datei. Die passende Zeile ist pro Aufgabe schon eingestellt. Nur in Aufgabe 16 wechselst du sie selbst.

Du arbeitest nur mit `$ausgabe`

```
$ausgabe = "";

// Hier kommt deine kleine PHP-Auswertung hin.
```

Der vorbereitete HTML-Bereich zeigt `$ausgabe` als Text an. Mit `"\n"` erzeugst du neue Zeilen; mit `.=` hängst du Text an. Du brauchst keine HTML-Tags in die Variable zu schreiben.

Die Ausgabe mit `htmlspecialchars` ist bereits fertig. Sonderzeichen werden als Text angezeigt. Auch kleine Typprüfungen und das Einlesen von Werten sind ab Aufgabe 05 teilweise vorgegeben; der Arbeitsauftrag nennt dir die verfügbaren Variablen.

Grundlage: Kapitel 6, S. 17–21; Kapitel 4, HTML Teil 2, S. 23–30. Neue, bewusst vereinfachte Aufgaben; technische Hinweise mit PHP-Handbuch abgeglichen. Quellen auf Seite 11.

Nur die wichtigsten Bausteine

Baustein	Bedeutung
<code>isset(\$daten["feld"])</code>	Schlüssel ist gesetzt und sein Wert ist nicht null. Ein leerer Text ergibt true.
<code>trim(\$text) === ""</code>	Text ist nach Entfernen äußerer Leerzeichen leer.
<code>\$wert = \$daten["feld"] ?? "";</code>	Fehlenden beziehungsweise null-Wert durch leeren Text ersetzen.
<code>is_string(\$wert)</code>	Prüft, ob der Wert Text ist. In mehreren Vorlagen schon eingebaut.
<code>\$ausgabe .= \$text . "\n";</code>	Text und einen Zeilenumbruch an die Ausgabe anhängen.
<code>break;</code>	Beendet hier switch oder die gerade betrachtete Schleife.

Stolperstelle: `empty` behandelt auch `"0"` als leer. Deshalb prüfen wir leere Texte in diesen Aufgaben ausdrücklich mit `trim($text) === ""`. Vorhandensein, Datentyp und erlaubter Wert sind unterschiedliche Prüfungen.

So prüfst du jede Aufgabe

- Einmal mit den angegebenen Standardwerten testen.
- Mindestens den zweiten Testfall vom Aufgabenblatt durchführen.
- Ausgabe mit der Erwartung vergleichen. Erst dann weitergehen.

GET: Query-Werte kommen in `$_GET` an. **POST:** Die üblichen Formular Daten im Request-Body kommen in `$_POST` an. POST allein ist keine Verschlüsselung. Nutze nur fiktive Daten.

Grundlagen und Quellen

Unterrichtsbezug: „PHP-Kurs – Kapitel 6: PHP DB“, S. 17–21; „Kapitel 4: HTML, Teil 2“, S. 23–30; die zuvor bereitgestellte PHP-Arrays-Webvariante. Diese Aufgabenserie, ihre vier Formularfelder und die konkreten Testfälle sind neu und gegenüber der Mehrseiten-Demo vereinfacht.

Formulardaten: www.php.net/manual/en/language.variables.external.php

Arrays: www.php.net/manual/de/language.types.array.php

isset / empty: www.php.net/manual/de/function.isset.php

foreach: www.php.net/manual/de/control-structures.foreach.php

switch: www.php.net/manual/de/control-structures.switch.php

HTML-Ausgabe: www.php.net/manual/de/function.htmlspecialchars.php